

ASD 2025 - II Appello - Programmazione

venerdì, 21 febbraio 2025, 12:35

1. (2 punti)

È dato un ADT `editori_adt`, in grado di rappresentare un elenco di editori (un vettore allocato dinamicamente). Ogni editore, oltre al nome (una stringa allocata dinamicamente), contiene un vettore di puntatori a libri (tipo `struct libro`): i libri sono da considerarsi come esterni alla struttura dati: l'ADT è quindi di tipo "aggregato", per quanto concerne i libri, che non vanno né allocati né deallocati. Sono date le definizioni dei tipi. Si scrivano due funzioni: una che duplica e una che libera l'ADT.

```
typedef struct {
    char *nome;
    struct libro **libri;
    int n_libri;
} editore_t;
```

```
struct editori {
    editore_t *editori;
    int n_editori;
};

typedef struct editori *editori_adt;
```

```
editori_adt EditoriDup(editori_adt e) {

}
```

```
void EditoriFree(editori_adt e) {

}
```

2. (4 punti)

È dato un BST (o albero binario, il problema è equivalente). Si scriva una funzione che verifichi se l'albero è bilanciato: si utilizzi come criterio di bilanciamento quello basato sulle dimensioni dei sotto-alberi. La funzione abbia prototipo:

```
int BSTisBalanced(BST b);
```

Si richiede, oltre alla funzione, la definizione del tipo BST e del tipo usato per il nodo. Si assuma che il nodo dell'albero NON sia stato esteso in modo da contenere la dimensione del relativo sotto-albero.

3. (6 punti)

È dato un vettore `V` contenente `N` interi non negativi: il vettore non è ordinato e sono possibili dati ripetuti. Si vogliono generare, come risultato, due vettori (allocati dinamicamente) `V1` e `V2`, in cui suddividere gli interi, in modo tale che, date la somma `S1` degli interi in `V1` e la somma `S2` degli interi in `V2`, si minimizzi $|S2-S1|$. La funzione deve poter essere chiamata come segue:

```
int N1, N2, *V1, *V2;
N1 = bestSplit(V, N, &V1, &V2);
N2 = N-N1;
```

`V1` e `V2` sono (puntatori a) vettori di interi che vanno allocati dinamicamente nella `bestSplit`, `N1` è il numero di interi in `V1`, mentre `N2` è il numero di interi in `V2`. Si riportano una soluzione (parziale) e le domande a cui rispondere.

```

void bestSolR(int *val, int *sol,
             int *bestSol, int sum,
             int *bestSumP,
             int goal, int n, int start) {
    int i;
    if (checkTerminal(n,start,sum,goal)) {
        checkBestSol(n, sol, bestSol, sum,
                    bestSumP, goal);
        return;
    }
    for (i = <1>; i < n; i++) {
        sol[i] = 1;
        bestSolR(<2>);
        sol[i] = 0;
    }
}

```

```

int bestSplit(<3>) {
    int *sol = calloc(N,sizeof(int));
    int *bestSol = calloc(N,sizeof(int));
    int bestSum=0;
    int goal = 0;
    int n1;
    for (int i=0; i<N; i++) goal += V[i];
    goal /= 2;
    // la funzione ricorsiva trova i dati per
    // uno dei due vettori; i dati non
    // selezionati andranno nel secondo vettore.
    bestSolR(0, V, sol, bestSol, 0,
            &bestSum, goal, N, 0);
    n1 = buildSol(V,N,bestSol,V1p,V2p);
    free(sol);
    free(bestSol);
    return n1;
}

```